



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

Every Path: Aplicación móvil

Integrantes:
Sara Alejandra Zambrana Taylor
Solieth Valentina Trejos Perez

Docente:
Jose Duran

Managua, 29 de junio del 2026

Índice

Índice	2
1. Introducción y objetivo de la entrega	3
2. Documentación técnica de la aplicación	3
2.1 Arquitectura general	3
2.2 Tecnologías utilizadas	3
2.3 Modelo de datos principal	4
2.4 Backend y API REST	4
3. Estructura de carpetas y archivos principales	5
4. Explicación de la solución desarrollada	6
4.1 Problema que resuelve	6
4.2 Solución propuesta	6
4.3 Funcionalidades principales	7
5. Flujo general de funcionamiento	7
5.1 Flujo de inicio de sesión	7
5.2 Flujo de visualización de metas	7
5.3 Flujo de progreso	8
6. Guía de demostración para la exposición	8
6.1 Preparación antes de exponer	8
6.2 Orden recomendado para la demostración	8
6.3 Guion breve para decir durante la demostración	9
7. Evidencias recomendadas durante la presentación	9
8. Limitaciones y mejoras futuras	10
9. Conclusión	10
10. Fuentes técnicas consultadas	10

1. Introducción y objetivo de la entrega

Everpath es una aplicación Android orientada al crecimiento personal. Su propuesta consiste en representar las metas del usuario como un mapa interactivo, donde cada objetivo, actividad y avance forma parte de una ruta personal. La aplicación busca alejarse del concepto tradicional de lista de tareas y presentar una experiencia visual basada en progreso, exploración y motivación.

El objetivo de esta entrega es cumplir tres puntos solicitados: presentar documentación técnica, explicar la solución desarrollada y realizar una demostración clara del funcionamiento. Por esa razón, este documento no solo describe la estructura del proyecto, sino que también resume cómo funciona la solución y qué pasos deben mostrarse durante la exposición.

Objetivo central

Mostrar que Everpath fue construida con una arquitectura ordenada, una solución funcional y un flujo demostrable ante el docente.

2. Documentación técnica de la aplicación

2.1 Arquitectura general

La aplicación Android está organizada bajo Clean Architecture y MVVM. Esta decisión permite separar la interfaz visual, la lógica de negocio y el acceso a datos. De esta forma, cada parte del sistema tiene una responsabilidad clara y el proyecto puede mantenerse y ampliarse con menos riesgo.

UI / Compose



ViewModel



UseCase



Repository



Room Database / API REST

La base local Room actúa como fuente principal de información en el dispositivo. La API REST se utiliza para autenticación, respaldo y sincronización con el servidor. Este enfoque permite que la aplicación conserve funcionalidad aun cuando no exista conexión a internet.

Capa	Responsabilidad	Ejemplos
Presentation	Muestra pantallas, componentes y estados visuales.	LoginScreen, EverpathScreen, GoalDetailScreen, ProfileScreen
Domain	Contiene modelos puros, contratos y casos de uso.	Goal, Activity, LoginUseCase, GoalRepository
Data	Gestiona Room, Retrofit, mappers, sesión y repositorios concretos.	GoalDao, RetrofitProvider, AuthRepositoryImpl
Navigation	Controla rutas y flujo entre pantallas.	AppNavHost, AppDestination, BottomNavigationBar
DI	Centraliza creación y entrega de dependencias.	AppContainer, módulos de red, base de datos y repositorios

2.2 Tecnologías utilizadas

Área	Tecnología	Uso dentro del proyecto
Lenguaje Android	Kotlin	Desarrollo de la aplicación móvil.
Interfaz	Jetpack Compose y Material 3	Construcción declarativa de pantallas y componentes visuales.
Navegación	Navigation Compose	Cambio entre Login, Everpath, detalle, perfil y demás pantallas.
Persistencia local	Room	Almacenamiento local de metas, actividades, posiciones y datos del usuario.
Comunicación HTTP	Retrofit y OkHttp	Consumo de endpoints REST del backend.
Asincronía	Coroutines y Flow	Operaciones suspendidas y lectura reactiva de datos.
Backend	Spring Boot	Exposición de API REST y reglas de negocio del servidor.

Área	Tecnología	Uso dentro del proyecto
Base de datos remota	PostgreSQL	Persistencia centralizada de usuarios y datos sincronizados.
Seguridad	JWT y BCrypt	Autenticación, autorización y protección de contraseñas.

2.3 Modelo de datos principal

El modelo central de Everpath se basa en metas llamadas GoalNodes. Cada GoalNode puede tener actividades asociadas, conexiones con otras metas, una posición visual dentro del mapa y un estado de avance. Las Activities representan las acciones concretas que generan progreso.

Entidad	Función dentro de Everpath
User	Representa al usuario autenticado, su nivel, experiencia y datos de perfil.
GoalNode / Goal	Representa una meta dentro del mapa de vida del usuario.
GoalConnection	Representa la relación o dependencia entre metas.
GoalPosition	Guarda la posición visual del nodo en el mapa.
Activity	Representa una acción concreta asociada a una meta.
Quest	Representa objetivos principales o experiencias secundarias.
Achievement	Representa logros desbloqueados por el progreso del usuario.
TimelineEvent	Registra eventos importantes dentro de la historia del usuario.

El progreso no debe tratarse como un dato manual. Se calcula a partir de las actividades completadas respecto al total de actividades de una meta. Esto mantiene la coherencia de la información y evita inconsistencias.

Progreso de una meta = actividades completadas / actividades totales

2.4 Backend y API REST

El backend está organizado en capas. Los controladores reciben peticiones HTTP, los servicios contienen reglas de negocio, los repositorios acceden a la base de datos y PostgreSQL conserva la información centralizada.

Controller
↓
Service
↓
Repository
↓
PostgreSQL

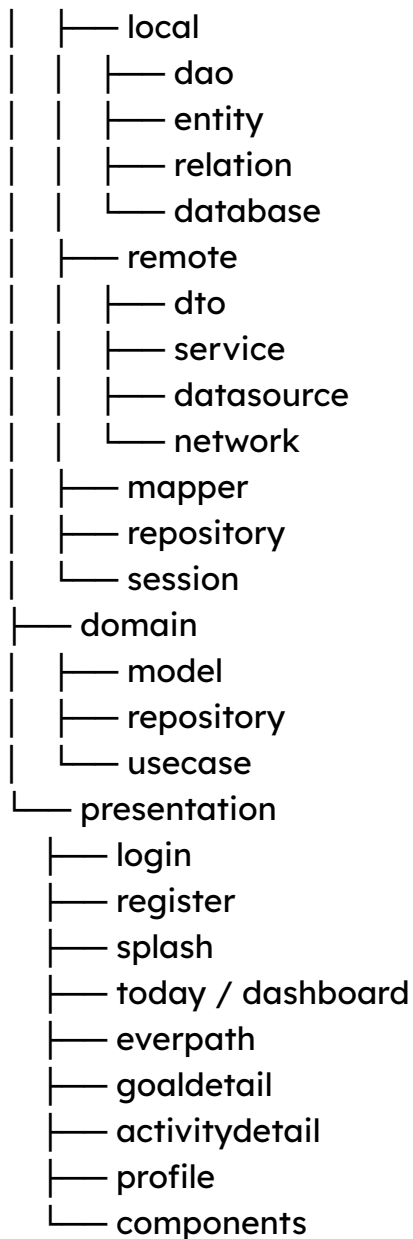
Módulo backend	Responsabilidad
config	Configuraciones generales, Swagger, CORS y seguridad.
security	Manejo de JWT, filtros de autorización y protección de endpoints.
controller	Recepción de peticiones REST y envío de respuestas.
service	Reglas de negocio y coordinación de operaciones.
repository	Acceso a datos mediante Spring Data JPA.
entity	Representación de tablas de la base de datos.
dto	Objetos de entrada y salida de la API.
mapper	Conversión entre entidades y DTOs.
exception	Manejo centralizado de errores.

3. Estructura de carpetas y archivos principales

La estructura de carpetas permite ubicar rápidamente cada responsabilidad del sistema. A continuación se resume la organización recomendada y usada para explicar el proyecto.

```

com.everpath
├── core
│   ├── constants
│   ├── extensions
│   ├── utils
│   ├── state
│   └── designsystem
├── di
├── navigation
└── data
  
```



Archivo o carpeta	Explicación clara
EverpathApplication.kt	Punto de arranque global de Android. Inicializa el contenedor de dependencias de la aplicación.
MainActivity.kt	Actividad principal. Carga el tema, Compose y el grafo de navegación.
AppContainer.kt	Centraliza la creación de dependencias como repositorios, casos de uso, base de datos, servicios y sesión.
AppNavHost.kt	Define las pantallas disponibles y las rutas de navegación.

Archivo o carpeta	Explicación clara
AppDestination.kt	Centraliza nombres de rutas para evitar strings repetidos o errores de escritura.
BottomNavigationBar.kt	Permite navegar entre secciones principales de la aplicación.
RetrofitProvider.kt	Construye la instancia de Retrofit para comunicación con la API.
ApiConstants.kt	Guarda la URL base y constantes relacionadas con la red.
NetworkClient.kt	Configura OkHttp, interceptores, headers y token cuando corresponde.
AuthApiService.kt	Define endpoints de login y registro.
SessionManager.kt	Guarda y recupera la sesión del usuario.
UserSession.kt	Mantiene los datos de la sesión activa en memoria.
GoalEntity.kt	Tabla local de metas en Room.
GoalPositionEntity.kt	Tabla local para guardar la posición de cada meta en el mapa.
GoalDao.kt	Consultas locales para insertar, leer, actualizar o eliminar metas.
GoalRepositoryImpl.kt	Implementa el acceso a metas combinando Room, API, mappers y reglas de sincronización.
LoginViewModel.kt	Coordina el proceso de inicio de sesión desde la pantalla.
EverpathScreen.kt	Pantalla principal del mapa de vida.
GoalDetailScreen.kt	Pantalla de detalle de una meta y sus actividades.

4. Explicación de la solución desarrollada

La solución desarrollada consiste en un MVP de Everpath que permite al usuario autenticarse, ingresar a la aplicación, visualizar su progreso y navegar por las secciones principales. El eje del sistema es el mapa Everpath, donde las metas se representan como nodos visuales asociados a actividades y progreso.

4.1 Problema que resuelve

Muchas aplicaciones de productividad muestran tareas como listas aisladas. Everpath busca resolver ese problema transformando las metas personales en una representación visual más significativa: un mapa de crecimiento. Así, el usuario no solo ve pendientes, sino una ruta de evolución personal.

4.2 Solución propuesta

Necesidad	Solución implementada en Everpath
Organizar metas personales	Uso de GoalNodes como unidades principales del mapa.
Visualizar progreso	Cálculo de progreso a partir de actividades completadas.
Mantener datos disponibles	Persistencia local con Room bajo enfoque Offline First.
Conectar con servidor	Comunicación mediante Retrofit hacia una API REST.
Proteger usuario	Autenticación con login, registro, sesión y token JWT.
Motivar al usuario	Sistema de XP, niveles, quests y logros básicos.
Facilitar navegación	AppNavHost y BottomNavigationBar para secciones principales.

4.3 Funcionalidades principales

- Inicio de sesión y registro de usuario mediante endpoints de autenticación.
- Restauración de sesión al iniciar la aplicación.
- Navegación principal entre las pantallas del MVP.
- Visualización del mapa Everpath como pantalla central del producto.
- Gestión de metas y actividades asociadas.
- Cálculo de progreso basado en actividades completadas.
- Persistencia local con Room para funcionamiento sin conexión.
- Comunicación con backend REST mediante Retrofit.
- Diseño visual con Jetpack Compose y Material 3.

5. Flujo general de funcionamiento

5.1 Flujo de inicio de sesión

```
LoginScreen
↓
LoginViewModel
↓
LoginUseCase
↓
AuthRepository
↓
AuthRemoteDataSource
```

↓
AuthApiService
↓
Backend /api/v1/auth/login
↓
SessionManager guarda la sesión
↓
Navegación a la pantalla principal

5.2 Flujo de visualización de metas

EverpathScreen
↓
EverpathViewModel
↓
GetGoalsUseCase
↓
GoalRepository
↓
Room Database
↓
GoalNodeCards en Compose

5.3 Flujo de progreso

Usuario completa una Activity
↓
ActivityRepository actualiza Room
↓
Se recalcula progreso del GoalNode
↓
La UI recibe el cambio mediante Flow
↓
Compose actualiza la pantalla automáticamente

6. Guía de demostración para la exposición

Esta sección sirve como guion para realizar una demostración clara ante el docente. Se recomienda mostrar primero la arquitectura de forma breve y luego ejecutar la aplicación para evidenciar su funcionamiento.

Duración sugerida

La demostración puede realizarse en 5 a 8 minutos. Lo importante es mostrar un flujo completo y no solo pantallas aisladas.

6.1 Preparación antes de exponer

- Abrir el proyecto Android en Android Studio.
- Abrir el backend en el IDE correspondiente, si será demostrado.
- Verificar que la URL base de la API apunte correctamente al servidor local o remoto.
- Ejecutar el backend y comprobar que no existan errores críticos.
- Ejecutar la aplicación Android en emulador o dispositivo físico.
- Tener preparado un usuario de prueba para login.

6.2 Orden recomendado para la demostración

1. Mostrar estructura del proyecto: Abrir Android Studio y señalar las carpetas presentation, domain, data, navigation, di y core. Explicar que cada capa tiene una responsabilidad distinta.
2. Mostrar el backend: Mostrar brevemente la estructura controller, service, repository, entity, dto y security. Explicar que el backend no maneja UI, solo API, seguridad y persistencia.
3. Ejecutar la app: Abrir la aplicación en el dispositivo o emulador y mostrar la pantalla inicial o splash.
4. Iniciar sesión: Ingresar credenciales de prueba y explicar que la app llama al backend, recibe una respuesta y guarda la sesión.
5. Mostrar navegación principal: Recorrer las secciones principales usando la barra inferior o el flujo de navegación implementado.
6. Mostrar Everpath: Abrir la pantalla del mapa y explicar que los nodos representan metas del usuario.
7. Abrir detalle de una meta: Seleccionar una meta y mostrar descripción, progreso y actividades asociadas.
8. Completar una actividad: Marcar una actividad como completada y explicar que el progreso se recalcula automáticamente.
9. Mostrar persistencia: Cerrar y abrir la app o cambiar de pantalla para evidenciar que los datos siguen disponibles desde Room.
10. Cerrar con explicación técnica: Resumir el flujo UI -> ViewModel -> UseCase -> Repository -> Room/API y mencionar mejoras futuras.

6.3 Guion breve para decir durante la demostración

Everpath está organizada por capas. La pantalla no se conecta directamente a la base de datos ni al backend; primero pasa por un ViewModel, luego por casos de uso y finalmente por repositorios. Esto permite mantener el código ordenado y facilitar pruebas. La aplicación trabaja bajo un enfoque Offline First: Room conserva los datos localmente y el backend se utiliza para autenticación, respaldo y

sincronización. La pantalla principal representa las metas como un mapa visual, y el progreso se calcula según las actividades completadas.

7. Evidencias recomendadas durante la presentación

Para que la demostración sea más clara, se recomienda presentar evidencias visibles del funcionamiento y no solo explicar el código.

Evidencia	Qué demuestra
Proyecto compilando	Que la aplicación puede ejecutarse correctamente.
Pantalla de login funcionando	Que existe integración con autenticación y sesión.
Navegación entre pantallas	Que el flujo de usuario está implementado.
Pantalla Everpath	Que la propuesta central del producto está visible.
Detalle de meta y actividades	Que existe relación entre metas y acciones concretas.
Cambio de progreso	Que la lógica de avance funciona.
Datos persistentes	Que Room conserva información localmente.
Backend o Swagger	Que la API REST está documentada o disponible.
Logcat o respuesta JSON	Que Android se comunica con el backend cuando corresponde.

8. Limitaciones y mejoras futuras

Como MVP académico, la solución se enfoca en demostrar la arquitectura, el flujo principal y las funciones esenciales. Algunas características pueden ampliarse en versiones posteriores.

Área	Mejora futura
Sincronización	Implementar una cola de sincronización más robusta y manejo avanzado de conflictos.
Testing	Aumentar pruebas unitarias en UseCases, ViewModels y Repositories.
UX	Agregar más animaciones, estados vacíos ilustrados y microinteracciones.
Achievements	Ampliar reglas automáticas de logros y recompensas.
Discover	Agregar recomendaciones basadas en reglas o preferencias del usuario.
Seguridad	Fortalecer expiración de token, refresh token y validaciones adicionales.

9. Conclusión

Everpath presenta una solución organizada y funcional para representar metas personales como un mapa de crecimiento. Su arquitectura separa correctamente la interfaz, la lógica de negocio, el acceso a datos y la comunicación con backend. Esta división facilita el mantenimiento, la escalabilidad y la comprensión del proyecto.

La demostración propuesta permite evidenciar el funcionamiento principal: inicio de sesión, navegación, visualización del mapa Everpath, consulta de metas, actividades, cálculo de progreso y persistencia local. Con esto se cumple el objetivo de presentar documentación técnica, explicar la solución desarrollada y demostrar claramente el funcionamiento del sistema.

10. Fuentes técnicas consultadas

- ADR-001 - Everpath Architecture Decision Record: arquitectura oficial, alcance MVP, patrones y decisiones técnicas.
- Documento 2 - Estructura Android y Modelo de Datos: organización de carpetas, entidades, relaciones Room y mappers.
- Documento 3 - Backend, API REST, Autenticación y Persistencia: arquitectura del servidor, endpoints, seguridad y base de datos.

- Documento 4 - Plan de Desarrollo, Roadmap de Implementación y Gestión del Proyecto: fases del MVP y priorización.
- Documento 5 - UI/UX, Design System y Experiencia de Usuario: diseño visual, componentes, navegación y experiencia deseada.